Design and Analysis of Algorithms

ASSIGNMENT 1

CO1: *Algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution, etc.*

Assignment Title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System (Total Marks - 50)

Duration:1week

Scenario — University Student Record Retrieval

A university maintains a large academic database containing student records, each identified by a unique Registration Number. Faculty and administrative staff frequently need to determine whether a particular student record exists and retrieve the corresponding information.

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You are required to investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Your solution must remain practical for both small and very large datasets. Any preprocessing performed on the records must be included in your efficiency analysis.

Design Challenge:

The university now needs to process 50,000 search requests per hour on a database containing 1,000,000 student records. New student records are added only once per day.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Design the Algorithm 5 Marks

Design and propose **at least three meaningfully different algorithmic strategies** for locating a given registration number.

The three approaches must represent **distinct algorithmic strategies**. Iterative and recursive implementations of the **same underlying algorithm will not be considered as two different approaches**.

Your proposed approaches must include:

- At least **one iterative approach**
- At least **one recursive approach**
- A clear explanation of the basic idea behind each approach
- Pseudocode or an appropriate flowchart for each approach
- A brief justification for selecting each approach for investigation
- Any preprocessing required by the approach, such as sorting or restructuring the data

You are expected to **design and select the approaches independently**. The appropriateness and diversity of the selected algorithmic strategies form part of the assessment.

Evidence Expected

Three meaningfully different algorithmic strategies, supported by pseudocode/flowcharts, identification of the design strategy, preprocessing requirements, and justification for selecting each approach.

ANSWER – TASK 1: Design the Algorithm

Problem model. The database stores n unique registration numbers (fixed-length keys, e.g. 8 digits). The input of every algorithm is the collection of n records and a search key K; the output is the position / record of K, or "not found". Records arrive in registration (arrival) order, so the raw file is **unsorted**. Three genuinely different design strategies are proposed, each exploiting a different property of the data:

	Approach A	Approach B	Approach C
Name	Sequential (linear) search	Binary search on a sorted array	Hash-table (chained) lookup
Design strategy	Brute force	Decrease-and-conquer (decrease by half) on pre-sorted data	Transform-and-conquer / space–time trade-off (hashing)
Style	Iterative	Recursive	Iterative (hash + chain walk)
Preprocessing	None	Sort the keys (merge sort, Θ(n log n))	Build hash table with m ≈ n buckets, Θ(n)
Extra memory	None, O(1)	None beyond sorted copy; O(log n) recursion stack	O(n) for buckets and chain pointers

Approach A – Sequential (linear) search [Brute force, iterative]

Basic idea. Scan the records one after another from the first to the last and compare each registration number with the key until a match is found or the records are exhausted. It uses no property of the data other than equality, so it works on the raw, unsorted file.

Preprocessing: none.

```
ALGORITHM LinearSearch(R[0..n-1], K)
  // R: array of registration numbers (unsorted), K: search key
  for i <- 0 to n-1 do
      if R[i] = K then return i      // record found
  return -1                          // record does not exist
```

Why investigate it? It is the natural baseline: simplest to code and verify, needs no sorting or extra memory, and is the best choice when the database is tiny or when only a handful of searches are made. It shows what the "do-nothing" cost is against which the preprocessing-based approaches must be justified.

Approach B – Binary search on a sorted array [Decrease-and-conquer, recursive]

Basic idea. If the keys are sorted, comparing K with the middle key tells us which half cannot contain K, so half of the remaining records are discarded after every comparison. The same question is then asked on the surviving half (a smaller instance of the same problem), which is naturally expressed recursively.

Preprocessing: sort the registration numbers once (merge sort, Θ(n log n)). Because new records come in batches, the new keys can later be sorted separately and merged into the sorted array in Θ(n + k).

```
ALGORITHM BinarySearch(S, K, low, high)   // S sorted; first call (S, K, 0, n-1)
  if low > high then return -1                 // base case: empty range
  mid <- floor((low + high) / 2)
  if K = S[mid] then return mid                // key comparison
  else if K < S[mid] then
        return BinarySearch(S, K, low, mid - 1)
  else  return BinarySearch(S, K, mid + 1, high)

PREPROCESSING: S <- MergeSort(R)              // Θ(n log n), done once
```

Why investigate it? It is the classic way to turn ordering into speed: the number of comparisons grows only logarithmically, it needs no additional memory, it has a guaranteed worst case, and it also supports ordered operations (range of registration numbers, neighbours). It is the obvious recursive candidate.

Approach C – Hash-table lookup with separate chaining [Transform-and-conquer, iterative]

Basic idea. Instead of searching, compute where the key must be. A hash function h(K) = (K × 2654435761) mod m maps every registration number to one of m buckets; each bucket stores the (short) list of keys that collide there. A search computes h(K) once and scans only that one chain.

Preprocessing: build the table by inserting all n keys (m ≈ n so the load factor α = n/m ≈ 1); Θ(n) expected time.

```
ALGORITHM HashBuild(R[0..n-1])                // m <- n buckets, each an empty list
  for i <- 0 to n-1 do
      append R[i] to bucket[ h(R[i]) ]        // h(K) = (K * 2654435761) mod m

ALGORITHM HashSearch(K)
  j <- h(K)
  for each key X in bucket[j] do
      if X = K then return X                  // key comparison inside the chain
  return -1
```

Why investigate it? The two approaches above still look at many keys (n or log n). Hashing is the only strategy that aims for an expected constant number of key comparisons independent of n, at the price of extra memory and a non-trivial build step. It is the natural candidate for a read-heavy, large database.

Diversity check. A uses no preprocessing and no structure (brute force); B restructures the data by ordering and shrinks the problem by a constant factor each step; C restructures the data by an address calculation and avoids comparing against most keys altogether. These are three different algorithm-design strategies, not variants of one algorithm (an iterative binary search would not be a separate approach from B).

TASK 2 – Identify the Basic Operation 2 Marks

For each of the proposed algorithms, identify the **basic operation** that predominantly determines its execution time.

Discuss:

- What the basic operation is
- Where it occurs in the algorithm
- Why it is appropriate to use this operation for efficiency analysis
- How the number of times it executes may change as the input size (n) increases

Do not simply name an operation. You are expected to **relate the selected operation to the behaviour of the algorithm**.

Evidence Expected

Correct identification and justification of the dominant/basic operation for each proposed algorithm.

ANSWER – TASK 2: Identify the Basic Operation

In all three algorithms the cost is dominated by **comparing a stored registration number with the search key (a key comparison)**. Everything else (index increment, computing mid, loop control) is performed at most a constant number of times per key comparison, so counting key comparisons captures the growth of running time. The operation is the same in kind, which also makes the three approaches directly comparable. The preprocessing steps have their own basic operations (listed in the last two rows).

Algorithm	Basic operation	Where it occurs	How its count changes with n
A – Linear	Comparison `R[i] = K`	Inside the for-loop; once per iteration (line 3 of the pseudocode)	Up to n executions: it grows linearly because every extra record is one extra iteration. Found at position i → i comparisons; absent → n.
B – Binary (recursive)	Comparison of K with `S[mid]`	Once in every recursive call (the three-way test =, <)	One comparison per call, and each call halves the range, so the count grows by only 1 when n doubles (≈ $\log_2$ n).
C – Hash	Comparison of K with a key X in the selected chain (hash computation h(K) is one fixed-cost step per search)	Inside the loop that walks bucket[h(K)]	Depends on the chain length, i.e. on the load factor α = n/m, not on n itself. With m ∝ n it stays ≈ 1 + α/2 as n grows.
B – preprocessing	Comparison of two keys in merge	Inside the merge step of merge sort	≈ n $\log_2$ n − n over all merge levels.
C – preprocessing	Insertion of a key into a bucket (hash + append)	Loop in HashBuild	Exactly n insertions.

Why this is the right quantity. (1) The comparison is performed in the innermost loop / recursive step, so its count is proportional to the total running time: $T(n) \approx c_{op} \cdot C(n)$. (2) It is independent of the programming language and machine, so the analysis can be validated by counting it experimentally (Tasks 8–10). (3) For the three algorithms it directly mirrors their behaviour: A compares against every record it passes; B compares only against the midpoints, which is why it needs so few; C compares only inside one short chain.

TASK 3 – Derive the Time-Complexity Function Mathematically 6 Marks

Derive the time-complexity function for each proposed approach using mathematical analysis.

Discuss:

- The input size (n)
- The number of times the basic operation executes
- How the operation count changes with (n)
- The mathematical expression representing the running time
- Any preprocessing cost involved

- The overall cost when preprocessing and searching are considered together

You must show the **mathematical reasoning leading to the complexity result**.

Merely stating that an algorithm is (O(n)), (O(\log n)), etc., will not receive full credit.

Evidence Expected

A clear mathematical derivation of the time-complexity function for each approach, including relevant operation counts and preprocessing costs.

ANSWER – TASK 3: Derive the Time-Complexity Function Mathematically

Input size: n = number of student records (registration numbers). Let C(n) denote the number of key comparisons (basic operation), c the (constant) cost of one comparison plus loop overhead, and q the number of searches performed after preprocessing.

Approach A – Linear search

If the key is at position i (1-based) the loop performs exactly i comparisons; if it is absent the loop performs n.

$$C_{best}(n) = 1, \quad C_{worst}(n) = n$$

Let p be the probability that the key is present, and assume each of the n positions is equally likely (probability p/n each). Then

$$C_{avg}(n) = \Sigma_{i=1}^{n}\, i \cdot (p/n) + n(1 - p) = (p/n) \cdot n(n+1)/2 + n(1 - p) = p(n+1)/2 + n(1 - p)$$

For a successful search (p = 1): $C_{avg}(n) = (n+1)/2$; for an unsuccessful one (p = 0): n. Hence the running time is T(n) = c · C(n) + c_0, which is a linear function of n:

$$T_{worst}(n) = c{\cdot}n + c_0 \in \Theta(n), \quad T_{avg}(n) = c(n+1)/2 + c_0 \in \Theta(n)$$

Preprocessing: none (P_A = 0). **Total for q searches:** T_A(n, q) = q · c(n+1)/2 on average, i.e. Θ(q·n). For n = 1,000,000 one average search needs (n+1)/2 = 500,000 comparisons.

Approach B – Binary search (with sorting as preprocessing)

Each call performs one comparison and then continues on at most ⌊ n/2⌋ elements. The size of the range therefore goes n → ⌊ n/2⌋ → ⌊ n/4⌋ → … → 1 → 0. The range is non-empty for exactly ⌊ $\log_2$ n⌋ + 1 successive sizes (because ⌊ n/2^j⌋ ≥ 1 ⇔j ≤ $\log_2$ n), so the number of comparisons in the worst case is

$$C_{worst}(n) = \lfloor \log_2 n \rfloor + 1$$

(e.g. n = 1,000 → 10, 10,000 → 14, 100,000 → 17, 1,000,000 → 20 comparisons). For the average successful search take $n = 2^k - 1$, whose decision tree is a perfect binary tree: level j contains 2^{j-1} keys, each needing j comparisons, so

$$C_{avg}(n) = (1/n)\, \Sigma_{j=1}^{k}\, j \cdot 2^{j-1} = ((k - 1){\cdot}2^k + 1) / n = (k - 1) + k/n \approx \log_2 (n+1) - 1$$

using $\Sigma\, j{\cdot}2^{j-1} = (k-1)2^k + 1$ and $2^k = n + 1$. (n = 1,000,000: 19.93 − 1 ≈ 18.9 comparisons.) The best case is 1 comparison (the key is the first midpoint).

Preprocessing: merge sort of n keys. The merge of two runs of total length L needs at most L − 1 comparisons, and there are ⌈ $\log_2$ n⌉ merge levels, each handling n elements, so $P_B(n) \le n\lceil \log_2 n\rceil - 2^{\lceil \log_2 n\rceil} + 1 \approx n \log_2 n \in \Theta(n \log n)$ ($\approx 2 \times 10^7$ for $n = 10^6$).

Total: $T_B(n, q) = P_B(n) + q{\cdot}C(n) \approx n \log_2 n + q \log_2 n = (n + q) \log_2 n \in \Theta((n + q) \log n)$.

Approach C – Hash table with separate chaining

Let m be the number of buckets and α = n/m the load factor (α ≈ 1 here). Under the standard assumption of simple uniform hashing, a key falls in each bucket with probability 1/m, so the expected chain length is α.

Unsuccessful search: the whole chain of the selected bucket is scanned: $E[C_{miss}] = \alpha$.

Successful search: the i-th inserted key is preceded in its chain by (i − 1)/m keys on average, so it is found after 1 + (i − 1)/m comparisons. Averaging over all n keys:

$$E[C_{hit}] = (1/n)\ \Sigma_{i=1}^{n}\ (1 + (i-1)/m) = 1 + (n-1)/(2m) = 1 + \alpha/2 - \alpha/(2n)$$

With m = n (α = 1) this is ≈ 1.5 comparisons, independent of n. Adding the constant cost of computing h(K): T_C(n) = c_h + c·(1 + α/2) ∈ Θ(1) expected. Worst case: all n keys collide in one bucket, C_{worst} = n, i.e. Θ(n).

Preprocessing: n insertions, each Θ(1) (compute h and append to the chain): P_C(n) = n·(c_h + c_a) ∈ Θ(n).

Total: T_C(n, q) = P_C(n) + q·(1 + α/2) ≈ n + 1.5q ∈ Θ(n + q) expected.

Overall cost (preprocessing + q searches)

Approach	Preprocessing	One search (avg)	One search (worst)	Total for q searches
A – Linear	0	(n+1)/2	n	Θ(q·n)
B – Sort + binary	Θ(n log n)	≈ $\log_2$ n − 1	⌊ $\log_2$ n⌋ + 1	Θ((n + q) log n)
C – Build hash + lookup	Θ(n)	1 + α/2 ≈ 1.5	n (all collide)	Θ(n + q) expected

Reading the table: the preprocessing of B and C is paid once, whereas A pays n/2 comparisons on **every** search. B beats A as soon as n log n + q log n < q·n/2, i.e. for q > ≈ 2 $\log_2$ n (for n = 10^6: about 40 searches by operation count). C beats B in the long run because its per-search term is constant, while its build cost is lower asymptotically (n versus n log n) but larger in practice (Task 9).

TASK 4 – Analyse Best, Average and Worst-Case Behaviour 4 Marks

Analyse the performance of each algorithm under different input conditions.

Discuss, wherever applicable:

- What input condition produces the **best case**
- What input condition produces the **average case**
- What input condition produces the **worst case**
- How the number of operations changes in each case
- Why these differences occur

Do not simply provide a table containing complexity values. You are expected to **explain the conditions that lead to each case**.

Evidence Expected

A justified best-, average- and worst-case analysis for the proposed approaches, supported by appropriate mathematical reasoning.

ANSWER – TASK 4: Analyse Best, Average and Worst-Case Behaviour

Approach A – Linear search

- **Best case – key is the first record scanned.** The very first comparison succeeds, so C = 1 regardless of n. This only depends on where the record happens to lie in the unsorted file.
- **Worst case – key is the last record, or is not in the database.** Every record must be inspected before the algorithm can conclude, so C = n. The "not found" case is the real worst case because the algorithm has no way of stopping early: nothing in the data tells it that the key cannot appear later.
- **Average case – key present, equally likely at any position.** The expected position is (1 + 2 + … + n)/n = (n+1)/2, so about half the file is scanned. If the key is absent with probability 1 − p the average rises to p(n+1)/2 + n(1−p) (Task 3). The average is therefore of the same order as the worst case: the operations scale linearly with n in all but the lucky case.

Approach B – Binary search on sorted data

- **Best case – key equals the middle element of the whole array.** The first probe succeeds: C = 1. The algorithm is lucky once per search, but only for one specific key out of n.
- **Worst case – key is absent, or is found only when the range has shrunk to one element.** Each comparison removes at most half of the range, so the largest number of calls happens when the key lies in the deepest level of the decision tree: $C = \lfloor \log_2 n \rfloor + 1$ (20 for 10^6 records).
- **Average case – key present, all keys equally likely.** In the decision tree half of the keys sit on the deepest level, a quarter one level above, and so on; most keys therefore need almost the maximum number of probes, giving $C_{avg} \approx \log_2 (n+1) - 1$ (18.9 for $n = 10^6$), only about one comparison less than the worst case. Unsuccessful searches take $\approx \log_2 n$ comparisons. The gap between best, average and worst is thus tiny compared with linear search: the behaviour is almost input-independent.

Approach C – Hash table

- **Best case – key is the first entry of its bucket (or the bucket is empty for a missing key).** One comparison for a present key (zero for a missing key falling in an empty bucket), independent of n.
- **Average case – keys spread evenly (uniform hashing).** Chain lengths follow a Poisson-like distribution with mean α, so a successful search costs 1 + α/2 ≈ 1.5 comparisons and an unsuccessful one α ≈ 1 comparison. The cost depends on the load factor only, so growing n with m ∝ n does not change it.
- **Worst case – many keys collide in the same bucket.** This happens with a poor hash function or a pathological key set (e.g. a modulus that interacts badly with structured registration numbers such as year + branch + serial). In the extreme all n keys form one chain and C = n, i.e. the hash table degenerates into linear search. With a good multiplicative hash the longest chain observed was only 6–10 keys for n = 1,000 …

1,000,000 (Task 9), so this case is rare in practice but it is the reason hashing has no deterministic guarantee.

Why the differences occur. A's cost depends on *where* the record is in an unordered list; B's cost depends on how many times the range can be halved, which depends only on n; C's cost depends on how evenly the hash function spreads the keys, i.e. on collisions. The table below summarises the number of comparisons per search (analytical):

Case	A – Linear	B – Binary	C – Hash (α = 1)
Best	1	1	1
Average (successful)	(n+1)/2	≈ $\log_2$ n − 1	≈ 1.5
Average (unsuccessful)	n	≈ $\log_2$ n	≈ 1
Worst	n	⌊ $\log_2$ n⌋ + 1	n (pathological)

TASK 5 – Perform Asymptotic Analysis 3 Marks

Express and compare the growth behaviour of your proposed algorithms using appropriate asymptotic notation.

Discuss:

- Appropriate use of (O), (\Omega) and/or (\Theta), wherever relevant
- How each algorithm behaves as (n) becomes very large
- How the growth rates of the proposed approaches differ
- What the growth behaviour means when the dataset increases from **1,000 to 1,000,000+ records**

You are expected to **interpret the asymptotic behaviour in the context of the given application**, rather than merely state the notation.

Evidence Expected

Correct asymptotic representation and a meaningful comparison of the growth behaviour of the proposed approaches.

ANSWER – TASK 5: Perform Asymptotic Analysis

Asymptotic classification

Algorithm	Search time	Preprocessing	Total, q searches
A – Linear	$O(n)$ overall; best case $\Omega(1)$; worst & average case $\Theta(n)$	None	$\Theta(q \cdot n)$
B – Binary	Worst & average $\Theta(\log n)$; best case $\Omega(1) \rightarrow O(\log n)$ overall	$\Theta(n \log n)$ (any comparison sort is also $\Omega(n \log n)$)	$\Theta((n + q) \log n)$
C – Hash	Average $\Theta(1)$ (expected); worst case $O(n)$; best $\Omega(1)$	$\Theta(n)$	$\Theta(n + q)$ expected

Justification of the notation. For A, $T(n) = c(n+1)/2$ satisfies $(c/2) \cdot n \leq T(n) \leq c \cdot n$ for all $n \geq 1$, so $T \in \Theta(n)$ for the average and worst cases; a single Θ is not valid for the algorithm as a whole because the best case is $\Theta(1)$, so we state $O(n)$ and $\Omega(1)$. For B, $\lfloor \log_2 n \rfloor + 1$ lies between $\log_2 n$ and $2 \log_2 n$ for $n \geq 2$, hence $\Theta(\log n)$. For C, the expected number of probes $1 + \alpha/2$ is a constant, hence $\Theta(1)$; the $O(n)$ worst case cannot be improved by this analysis because all keys may collide.

Growth rates

The growth rates are ordered $1 \prec \log n \prec n \prec n \log n$. For example $\lim_{n \to \infty} (\log_2 n)/n = 0$ (by L'Hôpital: $(1/(n \ln 2))/1 \rightarrow 0$), so $\log n = o(n)$: no constant factor in the linear algorithm can compensate for the gap at large n. The hash table's constant behaviour lies below log n in the same sense.

Interpretation for 1,000 → 1,000,000+ records

n	A: avg comparisons (n/2)	B: worst comparisons	C: avg comparisons	Growth vs n = 1,000
1,000	500	10	≈ 1.5	—
10,000	5,000	14	≈ 1.5	A × 10; B + 4; C unchanged
100,000	50,000	17	≈ 1.5	A × 100; B + 7; C unchanged
1,000,000	500,000	20	≈ 1.5	A × 1,000; B + 10; C unchanged

When the database grows a thousand-fold, linear search becomes a thousand times slower, which turns a 30 µs lookup into a ~36 ms lookup; the binary search merely needs 10 more comparisons (10 → 20, i.e. doubles); the hash lookup needs the same number of comparisons. Practically: A is acceptable only for the smallest database or occasional queries, whereas B and C remain practical at 10^6 records and beyond. Since each factor-of-ten growth in n costs B only ≈ 3.3 extra comparisons, B would still need only ≈ 30 comparisons for a database of a billion records, while A would need 5×10^8. The asymptotic costs of preprocessing (n log n for B, n for C) matter only through the number of searches q that are made afterwards, which is why Task 12 re-examines the design when q becomes large.

TASK 6 – Formulate the Recurrence Relation 3 Marks

For the recursive approach, formulate a recurrence relation directly from the behaviour of the algorithm.

Clearly identify:

- The input size of the original problem
- The size of the recursive subproblem
- The number of recursive calls
- The work performed outside the recursive call
- The base case
- How these components lead to the recurrence relation

The recurrence must be **derived from your algorithm**, rather than simply writing a known recurrence formula.

Evidence Expected

A correctly formulated recurrence relation with clear identification of its components and base case.

ANSWER – TASK 6: Formulate the Recurrence Relation

The recursive approach is Approach B (recursive binary search). The recurrence is read directly from the pseudocode of Task 1:

```
BinarySearch(S, K, low, high)
  if low > high then return -1                  // (1) base case: no element left
  mid <- floor((low + high)/2)                  // (2) work outside recursion: O(1)
  if K = S[mid] then return mid                 //     one key comparison
  else if K < S[mid] then return BinarySearch(S, K, low, mid-1)    // (3) ONE recursive call
  else return BinarySearch(S, K, mid+1, high)                       //     on one half only
```

Component	Identification from the algorithm
Size of original problem	n = high − low + 1, the number of elements in the current range
Size of the sub-problem	The left part has $\lfloor (n-1)/2 \rfloor$ elements, the right part $\lceil (n-1)/2 \rceil = \lfloor n/2 \rfloor$ elements. The larger one is $\lfloor n/2 \rfloor$, so the worst-case sub-problem has size $\lfloor n/2 \rfloor$ ($\approx n/2$).
Number of recursive calls	Exactly **one**: the algorithm continues in the left half or in the right half, never in both (the other half is discarded). Hence a = 1, b = 2.
Work outside the recursive call	Computing mid and comparing K with S[mid]: a constant amount of work, i.e. 1 key comparison ($\Theta(1)$).
Base case	n = 0 (low > high): returns −1 immediately with no comparison, C(0) = 0. (For n = 1 one comparison is made and the next call has n = 0, so C(1) = 1.)

Deriving the recurrence. The worst-case cost on n elements is the cost of the work done in this call (1 comparison) plus the cost of the single recursive call on the larger remaining part ($\lfloor n/2 \rfloor$ elements):

$$C(n) = C(\lfloor n/2 \rfloor) + 1 \quad \text{for } n \geq 1, \qquad C(0) = 0 \quad (\text{equivalently } C(1) = 1)$$

In terms of running time, with the constant work $\Theta(1)$ outside the recursion: $T(n) = T(\lfloor n/2 \rfloor) + \Theta(1)$, $T(0) = \Theta(1)$. Here a = 1 sub-problem, of size n/b with b = 2, and the non-recursive work is $f(n) = \Theta(1) = \Theta(n^0)$.

Preprocessing recurrence (for completeness). The sorting step is also recursive (merge sort): two sub-problems of size n/2 and a linear-time merge give $M(n) = 2M(\lfloor n/2 \rfloor) + \Theta(n)$, with M(1) = 0 comparisons. It is solved in Task 7.

TASK 7 – Solve the Recurrence Relation 4 Marks

Solve the recurrence relation obtained in Task 6 using an appropriate analytical method.

You may use:

- Substitution method
- Iteration method
- Master Theorem
- Another appropriate analytical technique

Discuss:

- Why the selected method is applicable
- The steps involved in solving the recurrence
- The resulting complexity
- What the result indicates about the behaviour of the recursive algorithm as (n) increases

You are expected to **show the analytical process**, rather than provide only the final complexity.

Evidence Expected

A complete recurrence solution using an appropriate method, together with justification of the method and interpretation of the result.

ANSWER – TASK 7: Solve the Recurrence Relation

Chosen methods. The recurrence $C(n) = C(\lfloor n/2 \rfloor) + 1$ has a single sub-problem, a halving size and constant extra work, so it is solved exactly by **backward substitution (iteration)** and the result is then verified in two ways: by **mathematical induction** (to handle the floor for all n, not only powers of 2) and by the **Master Theorem** (the recurrence has the form $T(n) = aT(n/b) + f(n)$ with $a = 1 \geq 1$, $b = 2 > 1$ and $f(n)$ polynomially bounded, so the theorem applies).

1. Backward substitution ($n = 2^k$)

Let $n = 2^k$ (so $k = \log_2 n$), $C(1) = 1$:

$$C(2^k) = C(2^{k-1}) + 1$$

$$= [C(2^{k-2}) + 1] + 1 = C(2^{k-2}) + 2$$

$$= C(2^{k-3}) + 3 = \ldots = C(2^{k-i}) + i$$

Stopping when the argument reaches the base case ($i = k$): $C(2^k) = C(1) + k = 1 + k$, i.e.

$$C(n) = \log_2 n + 1$$

2. Induction for arbitrary n

Claim: $C(n) = \lfloor \log_2 n \rfloor + 1$ for all $n \geq 1$. **Basis:** $n = 1$: $\lfloor \log_2 1 \rfloor + 1 = 1 = C(1)$. ✓ **Step:** assume the claim for all values smaller than n ($n \geq 2$). Then $C(n) = C(\lfloor n/2 \rfloor) + 1 = \lfloor \log_2 \lfloor n/2 \rfloor \rfloor + 1 + 1$. For $n \geq 2$, $\lfloor \log_2 \lfloor n/2 \rfloor \rfloor = \lfloor \log_2 n \rfloor - 1$ (halving lowers the binary logarithm by exactly one before flooring), so $C(n) = \lfloor \log_2 n \rfloor - 1 + 2 = \lfloor \log_2 n \rfloor + 1$. ■

3. Master Theorem check

For $T(n) = aT(n/b) + f(n)$ with $a = 1$, $b = 2$, $f(n) = \Theta(1)$: compare $f(n)$ with $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$. Since $f(n) = \Theta(n^{\log_b a})$, **Case 2** applies: $T(n) = \Theta(n^{\log_b a} \cdot \log n) = \Theta(\log n)$. This agrees with the exact solution.

Result and interpretation

$$C_{worst}(n) = \lfloor \log_2 n \rfloor + 1 \in \Theta(\log n)$$

Every doubling of the number of records adds exactly **one** comparison. Therefore going from 1,000 to 1,000,000 records (a factor $1{,}000 \approx 2^{10}$) increases the worst case from 10 to 20 comparisons only. The recursion depth is also $\lfloor \log_2 n \rfloor + 1$ (≈ 20 frames), so the stack use of the recursive version is negligible.

Preprocessing recurrence (merge sort)

$M(n) = 2M(n/2) + cn$, with $a = 2$, $b = 2$, $f(n) = \Theta(n)$. Since $n^{\log_2 2} = n = \Theta(f(n))$, Case 2 gives $M(n) = \Theta(n \log n)$. (Iteration: $M(n) = 2^i M(n/2^i) + i \cdot cn$; with $i = \log_2 n$ this is $n \cdot M(1) + cn \log_2 n$.) This confirms the $\Theta(n \log n)$ preprocessing cost used in Tasks 3 and 5.

TASK 8 – Implement and Test the Algorithms 4 Marks

Implement all the proposed algorithmic approaches and experimentally test their behaviour.

Use datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Ensure that:

- All algorithms produce the correct result
- The same or equivalent datasets are used for comparison
- Testing conditions are kept reasonably consistent
- Appropriate test cases are included
- The implementation corresponds to the algorithms analysed in the previous tasks

The purpose of implementation is **not merely to demonstrate working code**, but to generate evidence for validating your theoretical analysis.

Evidence Expected

Working implementations of the proposed algorithms together with a clearly described and reproducible testing procedure.

ANSWER – TASK 8: Implement and Test the Algorithms

Environment. Python 3.12 on a single-core Linux machine; timing with `time.perf_counter()`. All three algorithms were implemented exactly as given in Task 1 and instrumented to return the number of key comparisons together with the result (the comparison counter corresponds to the basic operation of Task 2).

```
def linear_search(records, key):              # Approach A (iterative)
    comps = 0
    for i in range(len(records)):
        comps += 1                            # basic operation
        if records[i] == key: return i, comps
    return -1, comps

def binary_search_rec(arr, key, low, high, c):   # Approach B (recursive); arr is sorted
    if low > high: return -1
    mid = (low + high) // 2
    c[0] += 1                                 # basic operation
    if arr[mid] == key: return mid
    if key < arr[mid]: return binary_search_rec(arr, key, low, mid - 1, c)
    return binary_search_rec(arr, key, mid + 1, high, c)

class HashTable:                              # Approach C (separate chaining, m = n)
    def __init__(self, n): self.m = max(8, n); self.b = [None] * self.m
    def h(self, k): return (k * 2654435761) % self.m
    def insert(self, k):
        j = self.h(k)
        if self.b[j] is None: self.b[j] = [k]
        else: self.b[j].append(k)
    def search(self, k):
        comps = 0; chain = self.b[self.h(k)]
        if chain is None: return -1, 0
        for x in chain:
            comps += 1                        # basic operation
            if x == k: return x, comps
        return -1, comps
```

Test procedure (reproducible)

- **Datasets:** n = 1,000; 10,000; 100,000; 1,000,000 unique 8-digit registration numbers generated by `random.sample(range(10_000_000, 99_999_999), n)` with a fixed seed (42), stored in arrival (unsorted) order. The same dataset is given to all three algorithms; B receives its sorted copy and C its hash table, and both preprocessing steps are timed separately.
- **Search workloads per size:** (i) 200 randomly chosen existing keys (average successful case); (ii) the first record (best case of A); (iii) the last record (worst case of A); (iv) 200 keys that are guaranteed to be absent (values below 10,000,000) for unsuccessful searches; (v) the middle element of the sorted array (best case of B). Seeds for query selection are fixed (7), so runs are repeatable.
- **Correctness tests:** for every size, 50 present and 50 absent keys were searched by all three algorithms; the assertions checked that a hit returns the position/key of the right record and a miss returns −1 for all three algorithms. Edge cases n = 0, 1, 2, 3, the smallest and largest key of the sorted array and keys below/above the key range were also checked. **All tests passed.**
- **Consistency of conditions:** all runs in one process, one after the other, no other workload, the same machine; time per search is the mean over many queries (linear

search at n ≥ 100,000 uses fewer queries because each takes milliseconds); comparisons are counted exactly, so they do not suffer from timing noise.

- **Run it:** `python search_algos.py` regenerates all tables of Task 9 (results are written to a JSON file).

TASK 9 – Record Experimental Performance 3 Marks

Collect and present experimental performance data for each algorithm across the different input sizes.

Record appropriate measures such as:

- Execution time
- Number of comparisons
- Number of key operations
- Input size
- Other relevant performance measures, where justified

Present the results using appropriate:

- Tables
- Graphs
- Performance plots

Ensure that the presentation allows the different approaches to be easily compared.

Evidence Expected

Systematically collected experimental data presented clearly using appropriate tables and/or graphs.

ANSWER – TASK 9: Record Experimental Performance

All values below are measured, not estimated. Time is the mean time of one search (in microseconds, µs, unless stated); "comps" is the mean number of key comparisons per search.

Table 1 – Average (successful) search: time per search and comparisons

n	A time (µs)	A comps	B time (µs)	B comps	C time (µs)	C comps
1,000	13.34	468.5	1.34	9.05	0.28	1.47
10,000	165	5,111.8	1.76	12.21	0.24	1.57
100,000	1,868	49,626.8	2.43	15.53	0.33	1.53
1,000,000	19,871	466,756.4	3.68	18.96	0.30	1.42

Table 2 – Best and worst case

n	A best (µs / comps)	A worst (µs / comps)	B best (µs / comps)	B worst: absent key (µs / comps)	C best (µs / comps)	C longest chain
1,000	0.24 / 1	31.31 / 1,000	0.22 / 1	1.19 / 9	0.22 / 1.00	6
10,000	0.16 / 1	332 / 10,000	0.20 / 1	1.55 / 13	0.18 / 1.00	8
100,000	0.17 / 1	3,963 / 100,000	0.24 / 1	2.14 / 16	0.22 / 1.00	9
1,000,000	0.19 / 1	36,139 / 1,000,000	1.86 / 1	2.55 / 19	0.21 / 1.00	10

Table 3 – Preprocessing time (seconds)

n	A: none	B: sort (Timsort/built-in `sorted`)	C: build hash table
1,000	0	0.0001	0.0003
10,000	0	0.0013	0.0028
100,000	0	0.0196	0.0280
1,000,000	0	0.286	0.966

(For B, the built-in `sorted` was used as the preprocessing step because the aim is to measure the cost of ordering the data; it is a Θ(n log n) comparison sort.)

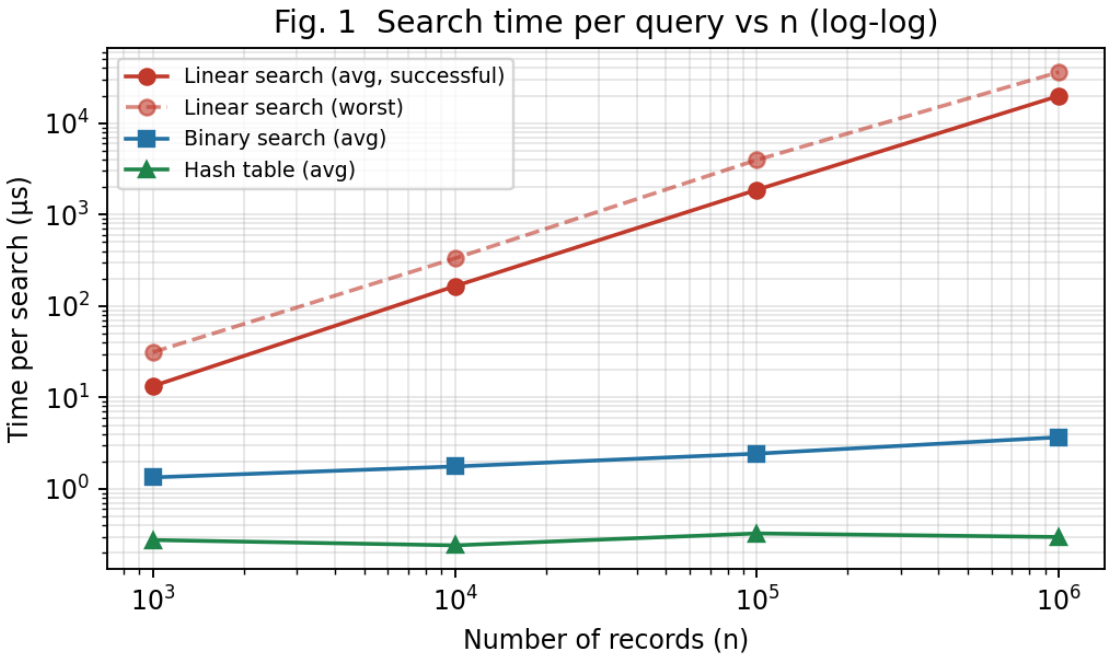


Figure 1 – Time per search versus n (log–log). Slopes: A ≈ 1, B small, C ≈ 0.

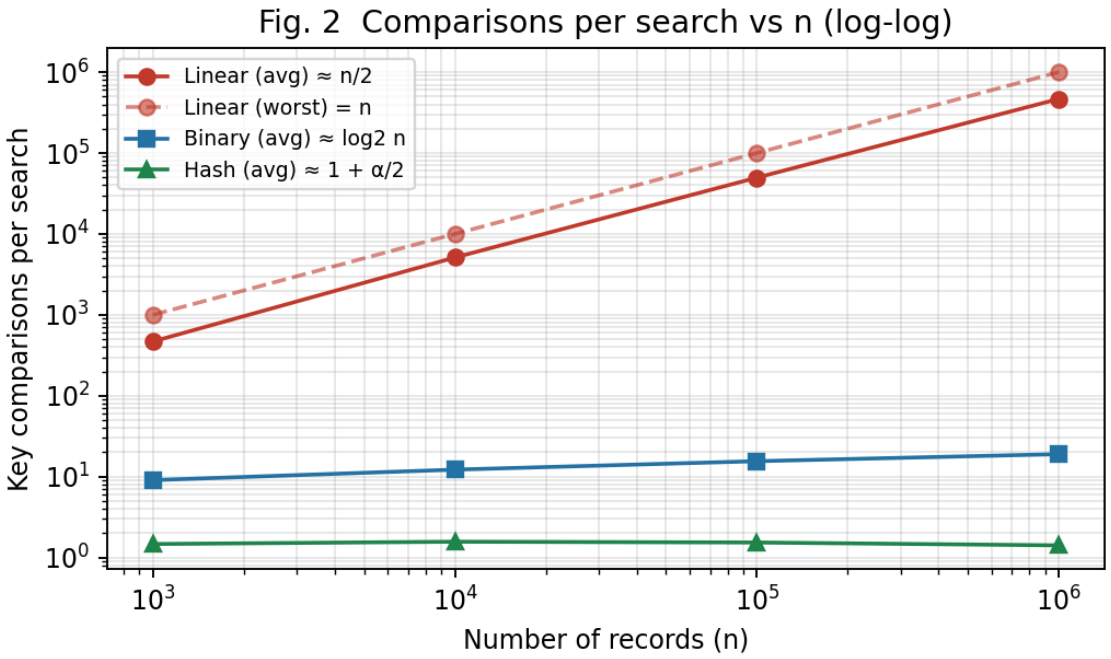


Figure 2 – Key comparisons per search versus n (log–log).

Observations. On the log–log plot linear search is a straight line of slope ≈ 1 (time ×1,490 for n ×1,000); binary search is almost flat (×2.7 for n ×1,000) and the hash lookup is practically horizontal (×1.1). The number of comparisons confirms this exactly: A needs ≈ n/2, B ≈ $\log_2$ n − 1, C ≈ 1.5.

Compare your theoretical analysis with your experimental results.

Discuss:

- Whether the observed growth agrees with the theoretical complexity
- Which algorithm scales better as (n) increases
- Any differences between theoretical prediction and actual execution
- Possible reasons for the differences

You are expected to **interpret the results**, rather than simply reproduce tables or graphs.

Evidence Expected

A reasoned comparison between theoretical and experimentally observed performance.

ANSWER – TASK 10: Compare Theory with Experimental Behaviour

Does the observed growth agree with the theory?

n	A: theory (n+1)/2	A: measured	B: theory (k−1)+k/n	B: measured	C: theory 1 + α/2	C: measured
1,000	500.5	468.5	8.98	9.05	1.50	1.47
10,000	5,000.5	5,111.8	12.29	12.21	1.50	1.57
100,000	50,000.5	49,626.8	15.61	15.53	1.50	1.53
1,000,000	500,000.5	466,756.4	18.93	18.96	1.50	1.42

- **Linear search.** The measured average comparisons (468.5; 5,112; 49,627; 466,756) lie within about 7% of n/2 (500; 5,000; 50,000; 500,000). The deviations are sampling error from using only 200 random keys (the standard deviation of a uniform position is ≈ n/√12, so the mean of 200 keys differs from n/2 by ≈ 2% of n). Worst-case counts are exactly n, as predicted. The growth is linear: every ×10 in n gives ≈ ×10 comparisons.
- **Binary search.** The measured averages (9.05; 12.21; 15.53; 18.96) match $\log_2 (n+1) - 1 + k/n$ (8.98; 12.29; 15.61; 18.93) to within 0.1 comparison, and unsuccessful searches reach the predicted $\lfloor \log_2 n \rfloor \approx 9, 13, 16, 19$ (the theoretical maximum is 10, 14, 17, 20). The recurrence solution $C(n) = \lfloor \log_2 n \rfloor + 1$ is therefore confirmed: +1 comparison for each doubling, +≈ 3.3 for each ×10.
- **Hash table.** The average number of comparisons stays at 1.4–1.6 for every n (theory 1.5), and unsuccessful searches cost ≈ 1.0 (theory α = 1), independent of n; the longest chain grows very slowly (6, 8, 9, 10), nowhere near the theoretical worst case of n. This confirms the expected Θ(1) behaviour and shows that the multiplicative hash spreads these keys uniformly.

Which algorithm scales better?

The hash table scales best, then binary search, then linear search. At n = 1,000,000 an average search takes 19.87 ms (A), 3.68 µs (B) and 0.30 µs (C): C is ≈ 66,649 times faster than A and ≈ 12 times faster than B; B is ≈ 5,404 times faster than A. The ordering is the same as the asymptotic ordering $\Theta(1) \prec \Theta(\log n) \prec \Theta(n)$.

Differences between prediction and measurement – and why

- **Time grows slightly faster than the comparison count.** For A, comparisons grow ×996 (468 → 466,756) but time grows ×1,490; the time per comparison rises from 28 ns to 43 ns. The reason is memory behaviour: at $n = 10^6$ the list and the Python integer objects no longer fit in the processor caches, so each element access is slower (cache misses). Asymptotic analysis counts operations and assumes each costs the same constant.
- **B's time grows ×2.7 while comparisons grow ×2.1.** The probes of a binary search jump across memory (the first probes are far apart), again causing cache misses at large n, and each probe in the recursive version also pays the overhead of a Python function call (≈ 19 calls per search at 10^6).
- **Constants hidden by Θ.** Per search, the hash table does a multiplication, a modulo and a list lookup in addition to the comparison, and B does a function call per level;

therefore, for n = 1,000, A with 468 comparisons costs only ≈ 13 µs and the three algorithms differ by tens of microseconds only. Asymptotic differences become decisive only for large n.

- **Measurement noise.** The 1-comparison best case of B at $n = 10^6$ took 1.86 µs against 0.2–0.24 µs for smaller n, although it makes the same single comparison; this is a timing artefact (single run, operating-system and cache effects), not a change in the algorithm. This is why the comparison count, not the clock, is used as the primary validation of the theory.
- **Preprocessing.** The measured costs (sort 0.286 s, hash build 0.966 s at $n = 10^6$) grow by ×2,759 and ×3,339 respectively when n grows ×1,000; the model predicts ×2,000 for n log n (= $10^6 \cdot 19.9 / 10^3 \cdot 10.0$) and ×1,000 for n. The measured ratios are higher for the same memory reasons (and because the 1,000-record times are tiny and hard to measure). In practice building the hash table costs about 3.4 times more than sorting: Θ(n) hides a bigger constant (hash computation, bucket and list-object allocation) than the highly optimised built-in sort.

TASK 11 – Recommend and Defend the Final Design 4 Marks

Based on your mathematical analysis and experimental findings, recommend the most appropriate algorithmic solution for the university student-record system.

Discuss:

- Which approach you recommend
- Why it is more appropriate than the alternatives
- Its theoretical efficiency
- Its experimentally observed performance
- Any preprocessing cost involved
- Relevant trade-offs or limitations
- Whether the solution remains practical as the database grows

Your recommendation must be based on **evidence obtained from your own analysis and experimentation**.

Simply identifying the algorithm with the lowest Big-O complexity is not sufficient.

Evidence Expected

A technically justified final design recommendation supported by both mathematical analysis and experimental evidence.

ANSWER – TASK 11: Recommend and Defend the Final Design

Recommendation (for the original problem, where the number of searches is not specified): Approach B – sort once, then search with binary search (recursive or iterative). Linear search is used only for very small collections; the hash table is the preferred alternative when the number of searches is known to be very large (examined in Task 12).

Why B is more appropriate than the alternatives

- **Theory.** Search cost Θ(log n) with a *guaranteed* worst case of $\lfloor \log_2 n \rfloor + 1 = 20$ comparisons at 10^6 records; preprocessing Θ(n log n) paid once; no extra memory beyond the sorted file and ≈ 20 stack frames.
- **Experiment.** At $n = 10^6$: 18.96 comparisons and 3.68 µs per search; sorting costs only 0.286 s. Versus A (19.87 ms per search, i.e. ≈ 5,404 times slower) the preprocessing is recovered after about 13 searches (0.286 s ÷ (21.6 ms − 3.7 µs) ≈ 13).
- **Versus C.** C is faster per search (0.30 µs versus 3.68 µs) but needs 0.966 s to build (3.4 × the sort), O(n) additional memory for buckets and chains, and gives only an *expected* Θ(1) with a worst case of n; it also keeps no ordering, so range or neighbour queries on registration numbers are impossible. Its extra preprocessing pays off only after ≈ 2×10^5 searches (0.97 − 0.29 = 0.68 s ÷ (3.68 − 0.30) µs).
- **Practicality as the database grows.** From 1,000 to 1,000,000 records B's time increases only ≈ 2.7 times and the number of comparisons from 9 to 19; a database of 10^9 records would need only ≈ 30 comparisons. A, in contrast, becomes impractical beyond ≈ 10^5 records (≈ 2–4 ms per search).

Trade-offs and limitations

- B requires the data to be kept sorted: inserting a record into a sorted array costs Θ(n); this is acceptable only if updates are rare or batched (sort the new keys and merge, Θ(n + k)).
- B's per-search time is bigger than C's by a constant factor ≈ 12 at 10^6 records and therefore C must be considered when searches are very frequent.
- A remains the best option for n of a few hundred and for one-off searches (no preprocessing); the experiment shows 13 µs at n = 1,000.
- Measurements are from an interpreted language on one machine; absolute times will differ in a compiled implementation, but comparison counts and growth rates do not.

Conclusion. The decision is based on the measured evidence (growth of comparisons, time per search, preprocessing costs and break-even number of searches), not only on the smaller Big-O: C has the smaller Big-O for searching, but for an unknown, moderate number of searches B gives the best total cost, the strongest guarantee and the lowest memory requirement.

Changed Requirement

The university now needs to process **50,000 search requests per hour** on a database containing **1,000,000 student records**.

However, new student records are added only **once per day**.

Re-evaluate your original design in response to these changed operating conditions.

You may:

RETAIN → MODIFY → or REDESIGN

your original solution.

A. Impact Analysis – 2 Marks

Discuss:

- What has changed from the original problem
- Which original assumptions are affected
- Why the frequency of searches is now important
- Why the relatively low frequency of record updates may influence the design

Evidence Expected

A clear analysis of how the changed requirements affect the original algorithmic design.

B. Evaluation of the Original Design – 2 Marks

Evaluate whether your original recommended solution remains appropriate.

Discuss:

- Its expected behaviour under 50,000 searches per hour
- Whether its preprocessing cost is acceptable under the new conditions
- Whether another approach could provide better overall performance

Evidence Expected

A technically supported evaluation of the suitability of the original design under the new operating conditions.

C. Adaptation or Redesign – 2 Marks

Decide whether to:

Retain, modify or redesign the original solution.

If you modify or redesign it:

- Describe the revised algorithmic strategy
- Explain what has changed
- Determine the effect on computational complexity

If you retain the original solution, provide sufficient technical justification for why no redesign is necessary.

Evidence Expected

An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence – 2 Marks

Defend your final design decision using evidence.

Use:

- Mathematical complexity analysis
- Experimental results, where appropriate
- Search frequency
- Preprocessing cost
- Dataset size
- Relevant computational trade-offs

Your final decision must follow logically from your analysis.

Evidence Expected

A well-supported final design decision integrating theoretical analysis, experimental evidence and the changed system constraints.

ANSWER – TASK 12: Re-evaluate and Adapt the Design

Decision in one line: MODIFY. The sorted array with binary search is kept as the ordered master copy and as the fallback, but the primary search path becomes a hash index that is rebuilt once a day. The reasoning follows.

A. Impact Analysis

- **What has changed.** The workload is now fully specified: n = 10^6 and q = 50,000 searches per hour (= 1.2 million per day, ≈ 13.9 searches per second), while the records change only once a day. Earlier the number of searches was unspecified and the preprocessing cost was weighed against a small number of searches.
- **Assumptions affected.** (1) That preprocessing is a significant fraction of the total cost: it is now spread over 1.2 million searches (sort 0.29 s ≈ 0.24 µs per search; hash build 0.97 s ≈ 0.8 µs per search). (2) That a Θ(n) search might be tolerable because few searches are made: it is not, since the total cost of linear search is q·n/2 ≈ 50,000 × 500,000 ≈ 2.3–2.5 × 10^{10} comparisons every hour. (3) That the structure must support frequent insertion: it does not.
- **Why search frequency is now important.** The total cost is P(n) + q·S(n). When q is small the preprocessing P(n) dominates and the cheapest design is the one with the smallest P; when q is large, S(n), the per-search cost, dominates and an expensive build is worthwhile. With q = 1.2 × 10^6 per day versus n = 10^6, we have q > n, i.e. the searches outweigh the database size itself.
- **Why rare updates matter.** The data are effectively static for 24 hours, so a structure can be built offline with a heavy-weight (but one-off) algorithm, and published atomically after the daily load. No dynamic structure (balanced trees, incremental re-hashing, locking during inserts) is needed; the cost of Θ(n) rebuilds is paid once per 1.2 million searches. The same property is what makes sorted arrays and hash tables, which are poor at online insertion, acceptable here.

B. Evaluation of the Original Design (sorted array + binary search)

Measured on n = 10^6 with 50,000 randomly chosen existing keys (one hour of traffic):

Strategy	Preprocessing (per day)	Time for 50,000 searches (1 h)	Time per search	Comparisons per hour	Per day (build + 24 h)
A – Linear	0	≈ 1,082 s (≈ 18 min), extrapolated from 300 timed queries	21.63 ms	≈ 2.3 × 10^{10}	≈ 25,960 s (≈ 7.2 h of CPU)
B – Sort + binary	0.286 s	0.268 s	5.36 µs	≈ 9.5 × 10^5	≈ 6.7 s
C – Hash index	0.966 s	0.0493 s	0.99 µs	≈ 7.5 × 10^4	≈ 2.1 s

- **Behaviour under 50,000 searches/hour.** B needs ≈ 0.27 s of processing per hour (≈ 0.007 % of the hour) and 20 comparisons at most per search; there is no capacity problem at all (≈ 270,000 searches/s possible against ≈ 14 required). So the original design **remains acceptable**.
- **Preprocessing cost.** Sorting 10^6 keys takes 0.29 s once per day (or a Θ(n + k) merge of the day's k new records into the sorted file): negligible relative to the 86,400 s day and to the 1.2 million searches it serves. It is acceptable.

- **Could another approach do better overall?** Yes, marginally in absolute terms but significantly in relative terms: the hash index answers 50,000 searches in 0.049 s versus 0.268 s (≈ 5.4 × faster), with ≈ 1.5 comparisons per search instead of 19. Its more expensive build (0.97 s versus 0.29 s) is recovered after about (0.966 − 0.286) s ÷ 4.4 µs ≈ 1.6×10^5 searches, i.e. in about three hours of normal traffic; over a day the hash design needs ≈ 2.2 s in total against ≈ 6.7 s. Linear search (A) is ruled out: ≈ 18 minutes of CPU every hour (≈ 30 % of a core permanently busy, 22–36 ms latency per search, ≈ 4,000 × slower than B).

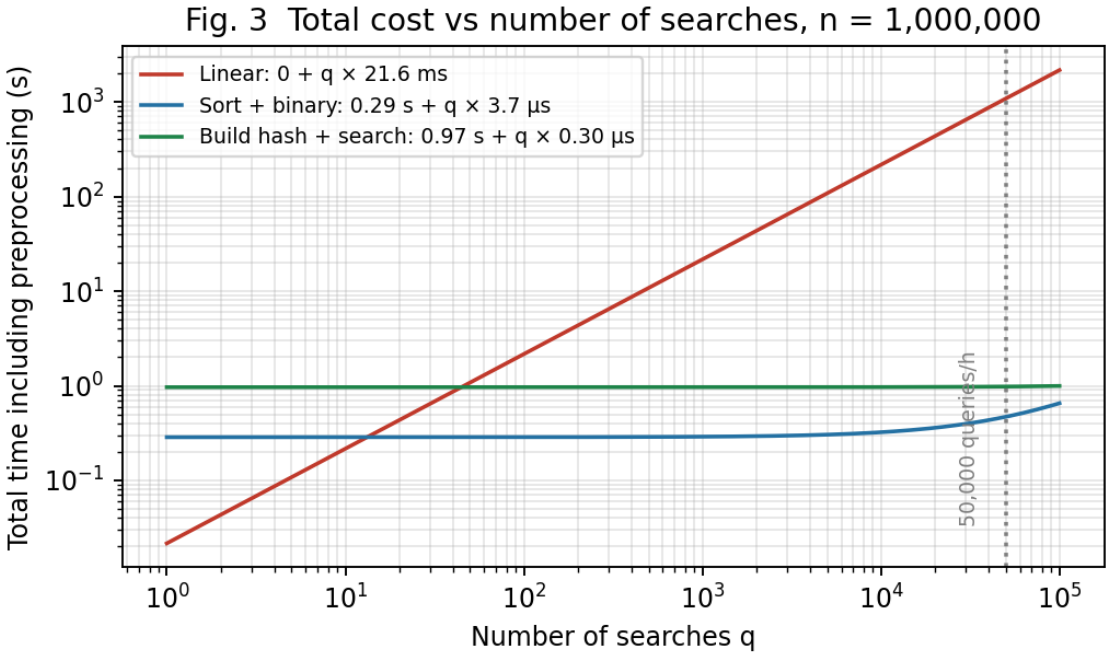


Figure 3 – Total time (preprocessing + q searches) at n = 1,000,000; the dotted line marks one hour of traffic.

C. Adaptation or Redesign – MODIFY

Revised strategy. Keep the data in a read-only, immutable form for 24 hours and serve point look-ups from a hash index; the sorted array is kept as a secondary structure.

```
DAILY BATCH (after the nightly load of the k new records):
   S <- Merge(S, MergeSort(newRecords))     // Θ(k log k + n): sorted master
   m <- smallest size >= n + k  (load factor α <= 1)
   H <- empty hash table with m buckets
   for each key X in S do insert X into H   // Θ(n + k)
   atomically replace the old index by H    // no half-built table is visible

SEARCH(K):
   chain <- H[ h(K) ]
   for each X in chain: if X = K return X   // expected ≈ 1.5 comparisons
   if |chain| > threshold: BinarySearch(S, K, 0, n-1)   // ≤ 20 comparisons
```

- **What changes.** The main search path moves from “compare along a halving range” to “calculate the address”; the daily build now consists of Θ(n) hashing plus an incremental merge of the day’s k new records into the sorted copy (Θ(n + k log k)) instead of a full re-sort; the structure is rebuilt in a daily batch, not updated online. A universal/multiplicative hash with load factor α ≤ 1 is used; the binary search remains

as a safety net so that the worst case never exceeds Θ(log n) even for an unlucky chain.

- **Effect on complexity.** Per search: Θ(log n) → Θ(1) expected (19 → ≈ 1.5 comparisons). Daily cost: Θ(n log n + q log n) → Θ(n + q) expected. With $n = 10^6$ and $q = 1.2 \times 10^6$: $\approx 2 \times 10^7 + 2.3 \times 10^7 \approx 4.3 \times 10^7$ basic operations versus $\approx 10^6 + 1.8 \times 10^6 = 2.8 \times 10^6$ for the modified design, about 15 times fewer. The memory requirement rises from O(1) extra to O(n) extra (one pointer/list per bucket), which for 10^6 records is a few tens of megabytes.
- **Why not retain unchanged?** Because the previous justification for B was "no extra memory, cheap preprocessing, unknown q"; for known, large q the lower per-search cost and the headroom are worth the build time and the memory, and updates do not require maintaining a dynamic structure.

D. Evidence-Based Defence

Evidence	Finding	Implication for the decision
Search frequency	50,000 / h = 1.2 M / day ≈ 14 per second	q > n; per-search cost dominates the total, not preprocessing
Dataset size	$n = 10^6$; linear needs $\approx 5 \times 10^5$ comparisons / search on average (21.6 ms measured)	Approach A rejected: ≈ 4,000 × slower than B and ≈ 22,000 × slower than C over the 50,000-search test
Complexity	A Θ(qn); B Θ((n+q) log n); C Θ(n+q) expected	C has the smallest total growth in q
Experiment (50,000 searches)	B: 0.268 s; C: 0.049 s; comparisons ≈ 19 vs ≈ 1.5	C is ≈ 5.4 × faster per search and 3 × cheaper over a day
Preprocessing	Sort 0.29 s; hash build 0.97 s (once per day)	Hash build is amortised in ≈ 3 h of traffic ($\approx 1.6 \times 10^5$ searches); ≈ 0.8 µs per search over the day
Update frequency	Daily batch only	Immutable index + atomic swap; no need for dynamic structures
Trade-offs	C: +O(n) memory, worst case n, no ordered queries	Mitigated by good hash function, load factor ≤ 1, chain-length threshold and the retained sorted array (B) for fallback and range queries

Final decision. The evidence shows that (i) linear search cannot sustain the load economically, (ii) the original binary-search design is correct and would be sufficient, but (iii) given a read-dominated workload of 1.2 million searches per day on a database that changes once a day, the hash index with a sorted-array fallback gives the lowest total cost (≈ 2.2 s versus ≈ 6.7 s per day), constant expected search time independent of future growth of n, and preserves a Θ(log n) worst-case guarantee. Therefore the design is **modified**, not entirely redesigned: the hashing layer is added in front of the sorted array and the daily rebuild is the only maintenance operation. If the university later needs registration-number range queries or must minimise memory, the binary-search design remains a technically sound choice.

MARK DISTRIBUTION

Task	Marks
Task 1 – Algorithm Design	5
Task 2 – Basic Operation	2
Task 3 – Mathematical Complexity Analysis	6
Task 4 – Best/Average/Worst-Case Analysis	4
Task 5 – Asymptotic Analysis	3
Task 6 – Recurrence Formulation	3
Task 7 – Recurrence Solution	4
Task 8 – Implementation & Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs Experiment	4
Task 11 – Final Design Recommendation	4
Subtotal	**42**
Task 12 – Day-4 Design Challenge	**8**
TOTAL	**50**